

Laboratory report of Nanjing University of Information Science and Technology Experiment

Title Lab 3 Date 2025.5.5 Class 24AC
Name ZhuangZhong ID 202483910032

1. Objectives

The main objective of this lab is to understand and practice fundamental Kubernetes operations using Minikube, including:

Installing and starting a Minikube local Kubernetes cluster

Deploying applications using Deployments

Viewing and managing Pods and Deployments

Exposing applications to external access using NodePort services

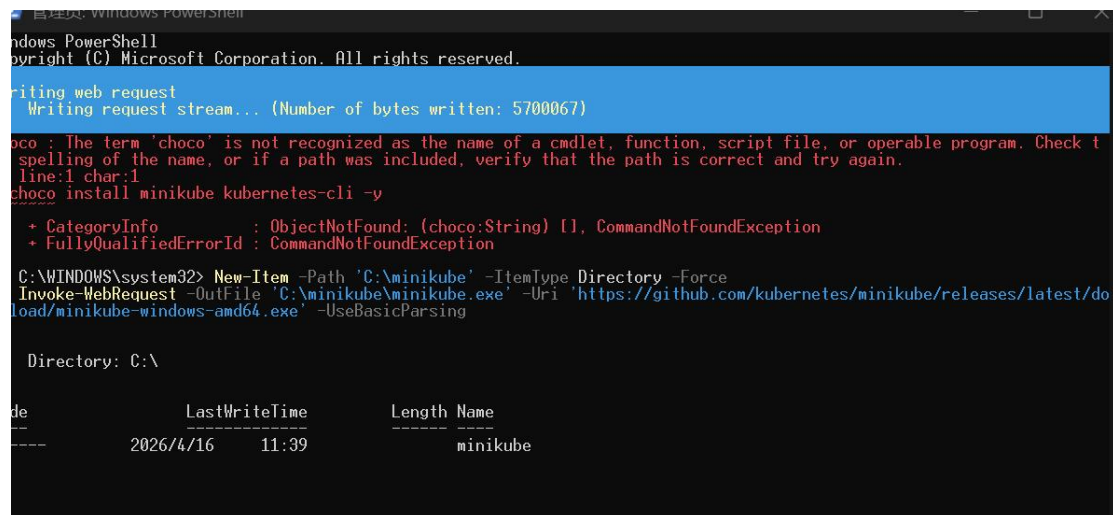
Accessing applications through service URLs

Troubleshooting common issues such as image pull failures and service unavailability

Understanding key Kubernetes concepts including Pods, Deployments, Services, and NodePort

2. Environment Preparation

Minikube and kubectl were installed on Windows 11. Docker Desktop was used as the container runtime driver. The installation was performed manually by downloading the Minikube executable from GitHub releases because Chocolatey was not available.



```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\> choco install minikube kubernetes-cli -y
Writing web request
Writing request stream... (Number of bytes written: 5700067)
choco : The term 'choco' is not recognized as the name of a cmdlet, function, script file, or operable program. Check the
spelling of the name, or if a path was included, verify that the path is correct and try again.
line:1 char:1
choco install minikube kubernetes-cli -y
+ ~~~~~
+ CategoryInfo          : ObjectNotFound: (choco:String) [], CommandNotFoundException
+ FullyQualifiedErrorId : CommandNotFoundException

PS C:\WINDOWS\system32> New-Item -Path 'C:\minikube' -ItemType Directory -Force
Invoke-WebRequest -OutFile 'C:\minikube\minikube.exe' -Uri 'https://github.com/kubernetes/minikube/releases/latest/download/minikube-windows-amd64.exe' -UseBasicParsing

Directory: C:\

Name                LastWriteTime         Length Name
----                -
minikube             2026/4/16 11:39             4 minikube
```

After installation, the versions were verified to ensure the environment was correctly set up. Minikube version v1.38.1 and kubectl client version v1.34.1 were confirmed to be working properly.

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\WINDOWS\system32> minikube version
>> kubectl version --client
minikube version: v1.38.1
commit: c93a4cb9311efc66b90d33ea03f75f2c4120e9b0
Client Version: v1.34.1
Kustomize Version: v5.7.1
PS C:\WINDOWS\system32>
```

3. Experiment Implementation and Results

3.1 Starting the Minikube Cluster

The command `minikube start` was executed to start a local Kubernetes cluster. The system automatically selected the Docker driver and began downloading the necessary Kubernetes preloaded files. The download included the `kiabase` image (approximately 1.28 GiB) which serves as the base image for the Minikube node.

```
PS C:\WINDOWS\system32> minikube version
> kubectl version --client
minikube version: v1.38.1
commit: c93a4cb9311efc66b90d33ea03f75f2c4120e9b0
Client Version: v1.34.1
Kustomize Version: v5.7.1
PS C:\WINDOWS\system32> minikube start
Microsoft Windows [11 Home 25H2] 上的 minikube v1.38.1
自动选择 docker 驱动。其他选项: hyperv, ssh
Starting v1.39.0, minikube will default to "containerd" container runtime. See #21973 for more info.
使用具有 root 权限的 Docker Desktop 驱动程序
在集群中 "minikube" 启动节点 "minikube" primary control-plane
正在拉取基础镜像 v0.0.50 ...
正在下载 Kubernetes v1.35.1 的预加载文件...
> preloaded-images-k8s-v18-v1...: 17.57 MiB / 272.45 MiB 6.45% 2.82 MiB p/s
```

During the startup process, Minikube successfully downloaded the required images. In case of download issues, Minikube automatically fell back to an alternative image source. After the image download completed, Kubernetes v1.35.1 was prepared in Docker.

```
base page via HTTP.
您很有可能遇到了网络问题。请确保您至少可以通过 HTTP（直连或代理）访问互联网。当前您的代理配置为:
> kibase-v0.0.50-amd64.tar: 1.28 GiB / 1.28 GiB 100.00% 2.64 MiB p/s 8m1
minikube was unable to download gcr.io/k8s-minikube/kibase:v0.0.50, but successfully downloaded kibase/stable:v0.0.50 as a fallback image
正在更新运行中的 docker "minikube" container ...
正在 Docker 29.2.1 中准备 Kubernetes v1.35.1a-
正在验证 Kubernetes 组件...
- 正在使用镜像 gcr.io/k8s-minikube/storage-provisioner:v5
启用插件: storage-provisioner, default-storageclass
完成! kubectl 现在已配置, 默认使用 "minikube" 集群和 "default" 命名空间
PS C:\WINDOWS\system32>
```

The cluster startup completed successfully with the storage-provisioner and default-storageclass addons enabled. `kubectl` was configured to use the "minikube" cluster and the "default" namespace by default.

```
在集群中 minikube 启动节点 minikube primary control-plane
* 正在拉取基础镜像 v0.0.50 ...
* 正在更新运行中的 docker "minikube" container ...
* 正在 Docker 29.2.1 中准备 Kubernetes v1.35.1a
* 正在验证 Kubernetes 组件...
  - 正在使用镜像 gcr.io/k8s-minikube/storage-provisioner:v5
* 启用插件: default-storageclass, storage-provisioner
* 完成! kubectl 现已配置, 默认使用"minikube"集群和"default"命名空间
error: failed to create deployment: deployments.apps "hello-minikube" already exists
Error from server (AlreadyExists): services "hello-minikube" already exists

X 因 SVC_UNREACHABLE 错误而退出: service not available: no running pod for service hello-minikube found
```

3.2 Creating a Deployment (hello-minikube)

An attempt was made to create a deployment named hello-minikube using the echoserver image with the command `kubectl create deployment hello-minikube --image=registry.k8s.io/echoserver:1.4`. However, the system returned an error indicating that the deployment already existed from a previous lab session.

```
正在更新运行中的 docker "minikube" container ...
正在 Docker 29.2.1 中准备 Kubernetes v1.35.1a
正在验证 Kubernetes 组件...
  - 正在使用镜像 gcr.io/k8s-minikube/storage-provisioner:v5
* 启用插件: storage-provisioner, default-storageclass
* 完成! kubectl 现已配置, 默认使用"minikube"集群和"default"命名空间
$ C:\WINDOWS\system32> kubectl create deployment hello-minikube --image=registry.k8s.io/echoserver:1.4
error: failed to create deployment: deployments.apps "hello-minikube" already exists
$ C:\WINDOWS\system32> kubectl expose deployment hello-minikube --type=NodePort --port=8080
service/hello-minikube exposed
$ C:\WINDOWS\system32> minikube service hello-minikube
```

NAMESPACE	NAME	TARGET PORT	URL
default	hello-minikube	8080	http://192.168.49.2:31129

```
因 SVC_UNREACHABLE 错误而退出: service not available: no running pod for service hello-minikube found

* 如果上述建议无法帮助解决问题, 请告知我们:
  https://github.com/kubernetes/minikube/issues/new/choose

* 请运行 minikube logs --file=logs.txt 命令, 并将生成的 logs.txt 文件附加到 GitHub 问题中。
* 请同时将以下文件附加到 GitHub 问题中:
  * - C:\Users\联想\AppData\Local\Temp\minikube_service_69987dfbfe442889617cd441ceddb8c1f5159959_0.log
```

The existing deployment was then exposed as a NodePort service using `kubectl expose deployment hello-minikube --type=NodePort --port=8080`. The service was successfully created, but when attempting to access it using `minikube service hello-minikube`, an `SVC_UNREACHABLE` error occurred because no running pod was available for the service. This indicated that the existing hello-minikube pod was not in a running state.

```
正在更新运行中的 docker "minikube" container ...
正在 Docker 29.2.1 中准备 Kubernetes v1.35.1a
正在验证 Kubernetes 组件...
  - 正在使用镜像 gcr.io/k8s-minikube/storage-provisioner:v5
* 启用插件: storage-provisioner, default-storageclass
* 完成! kubectl 现已配置, 默认使用"minikube"集群和"default"命名空间
$ C:\WINDOWS\system32> kubectl create deployment hello-minikube --image=registry.k8s.io/echoserver:1.4
error: failed to create deployment: deployments.apps "hello-minikube" already exists
$ C:\WINDOWS\system32> kubectl expose deployment hello-minikube --type=NodePort --port=8080
service/hello-minikube exposed
$ C:\WINDOWS\system32> minikube service hello-minikube
```

NAMESPACE	NAME	TARGET PORT	URL
default	hello-minikube	8080	http://192.168.49.2:31129

```
因 SVC_UNREACHABLE 错误而退出: service not available: no running pod for service hello-minikube found

* 如果上述建议无法帮助解决问题, 请告知我们:
  https://github.com/kubernetes/minikube/issues/new/choose

* 请运行 minikube logs --file=logs.txt 命令, 并将生成的 logs.txt 文件附加到 GitHub 问题中。
* 请同时将以下文件附加到 GitHub 问题中:
  * - C:\Users\联想\AppData\Local\Temp\minikube_service_69987dfbfe442889617cd441ceddb8c1f5159959_0.log
```

3.3 Creating a New Deployment (k8s-demo)

To resolve the pod availability issue, a new deployment named k8s-demo was created using the reliable nginx image with the command `kubectl create deployment k8s-demo --image=nginx`.

```

PS C:\WINDOWS\system32> kubectl create deployment k8s-demo --image=nginx
>> kubectl get deployments
>> kubectl get pods
deployment.apps/k8s-demo created
NAME          READY   UP-TO-DATE   AVAILABLE   AGE
hello-minikube 0/1     1            0           21m
k8s-demo      0/1     1            0           1s
NAME          READY   STATUS              RESTARTS   AGE
hello-minikube-84c4948447-m18kk 0/1     ImagePullBackOff    0           21m
k8s-demo-8b98749b7-6vd7i        0/1     ContainerCreating   0           1s

```

After creating the deployment, kubectl get deployments showed two deployments: hello-minikube (0/1 ready) and k8s-demo (0/1 ready). kubectl get pods revealed that the hello-minikube pod was in ImagePullBackOff status (indicating failure to pull the echoserver image), while the k8s-demo pod was in ContainerCreating status.

3.4 Exposing and Accessing the k8s-demo Service

The k8s-demo deployment was exposed as a NodePort service using kubectl expose deployment k8s-demo --type=NodePort --port=80. After exposing the service, kubectl get svc was used to view all services in the default namespace. The output showed three services: hello-minikube (NodePort on port 31129), k8s-demo (NodePort on port 30475), and the default kubernetes ClusterIP service.

```

PS C:\WINDOWS\system32> kubectl expose deployment k8s-demo --type=NodePort --port=80
>> kubectl get svc
>> minikube service k8s-demo --url
service/k8s-demo exposed
NAME          TYPE          CLUSTER-IP      EXTERNAL-IP      PORT(S)          AGE
hello-minikube NodePort      10.106.140.111  <none>           8080:31129/TCP   6m47s
k8s-demo      NodePort      10.106.13.106   <none>           80:30475/TCP     0s
kubernetes    ClusterIP     10.96.0.1       <none>           443/TCP          23m
http://127.0.0.1:64300

```

The access URL for the k8s-demo service was obtained using minikube service k8s-demo --url, which returned http://127.0.0.1:64300. This URL allows external access to the nginx web server running inside the Kubernetes cluster.

3.5 Verifying Nginx Service Functionality

Opening the obtained URL (http://127.0.0.1:64300) in a web browser displayed the nginx welcome page, confirming that the service was successfully deployed and accessible. The welcome page indicated that the nginx web server was successfully installed and working, with links to online documentation and commercial support.

Welcome to nginx!

If you see this page, nginx is successfully installed and working. Further configuration is required for the web server, reverse proxy, API gateway, load balancer, content cache, or other features.

For online documentation and support please refer to nginx.org.
To engage with the community please visit community.nginx.org.
For enterprise grade support, professional services, additional security features and capabilities please refer to f5.com/nginx.

Thank you for using nginx.

A simpler version of the nginx welcome page was also observed, further confirming that the service was functioning correctly.

Welcome to nginx!

If you see this page, the nginx web server is successfully installed and working. Further configuration is required.

For online documentation and support please refer to nginx.org.
Commercial support is available at nginx.com.

Thank you for using nginx.

4. Issues and Solutions

Three main issues were encountered during the experiment, along with their respective solutions.

Issue	Cause	Solution
choco command not found during installation	Chocolatey package manager was not installed on the system	Manually downloaded the Minikube executable directly from GitHub releases instead of using Chocolatey
ImagePullBackOff error for hello-minikube pod	The echoserver image (registry.k8s.io/echoserver:1.4) could not be pulled due to network or registry issues	Created a new deployment using the more reliable and widely available nginx image
SVC_UNREACHABLE error when accessing hello-minikube service	The service was exposed before any pod was in the running state; the existing pod was in ImagePullBackOff status	Checked pod status first using <code>kubectl get pods</code> , then created a new deployment with a working image, and exposed it only after the pod was running

Additionally, it is important to note that the error deployment already exists indicates that a resource with the same name already exists in the namespace. This can be resolved by either deleting the existing resource using `kubectl delete deployment <name>` or by using a different name for the new deployment.

5. Key Concepts Summary

The core Kubernetes operations covered in this lab include:

Operation	Command	Description
Start Minikube cluster	<code>minikube start</code>	Initializes a local Kubernetes cluster
Create a Deployment	<code>kubectl create deployment <name> --image=<image></code>	Deploys an application using a specified container image

View Deployments	<code>kubectl get deployments</code>	Lists all deployments in the current namespace
View Pods	<code>kubectl get pods</code>	Lists all pods and their status
View Services	<code>kubectl get svc</code>	Lists all services and their types
Expose a Service	<code>kubectl expose deployment <name> --type=NodePort --port=<port></code>	Creates a NodePort service to expose the deployment externally
Get Service URL	<code>minikube service <name> --url</code>	Retrieves the access URL for a service
Access a Service	<code>minikube service <name></code>	Opens the service in the default web browser

Key Kubernetes Concepts Explained:

Pod: The smallest deployable unit in Kubernetes, containing one or more containers that share storage and network resources.

Deployment: A controller that manages a set of identical Pods, ensuring the desired number of replicas are running and handling rolling updates.

Service: An abstraction that defines a logical set of Pods and a policy to access them, providing stable network endpoints.

NodePort: A service type that exposes the service on a static port on each Node's IP address, allowing external traffic to access the service.

6. Conclusion

This lab successfully demonstrated the core Kubernetes operations of cluster startup, application deployment, service exposure, and external access using Minikube. The following key achievements were accomplished:

Minikube cluster setup was completed successfully on Windows 11 using Docker Desktop as the driver, providing a local Kubernetes environment for learning and testing.

Deployment creation using both the echoserver and nginx images illustrated how to deploy containerized applications to a Kubernetes cluster.

Service exposure via NodePort enabled external access to applications running inside the cluster, demonstrating how Kubernetes handles network traffic routing.

Troubleshooting skills were developed by encountering and resolving common issues including ImagePullBackOff, SVC_UNREACHABLE, and naming conflicts.

Pod status monitoring proved to be critical—exposing a service before the underlying pod is running results in service unavailability.

The experience gained from this lab establishes a solid foundation for more advanced Kubernetes topics, including:

Using Ingress controllers for HTTP routing and load balancing

Managing configuration using ConfigMaps and Secrets
Scaling applications using `kubectl scale`
Performing rolling updates using `kubectl set image`
Working with PersistentVolumes for stateful applications
Multi-container pod design patterns